UNIT 4

Q1) Describe Bayesian network in short for learning and inferences.

A **Bayesian Network** (BN), also known as a Belief Network or Bayesian Belief Network, is a probabilistic graphical model that represents a set of variables and their conditional dependencies using a directed acyclic graph (DAG). It is widely used for reasoning under uncertainty, making predictions, and learning relationships between variables.

Key Components of a Bayesian Network

Nodes:

Represent random variables in the domain.

Each node is associated with a probability distribution.

Edges:

Directed edges between nodes represent conditional dependencies.

An edge from A to B means A influences B.

Conditional Probability Tables (CPTs):

Each node has a CPT that quantifies the relationship between the node and its parent nodes.

Example with Diagram

Consider a Bayesian Network for predicting traffic delays based on weather and accidents:

Nodes: Weather (WWW), Traffic (TTT), Accident (AAA)

Edges: W→T, A→T

CPTs: P(T|W, A), P(W), P(A)

Example CPT for Traffic:

Weather	Accident	Traffic (T) = Yes	Traffic (T) = No
Sunny	No	0.2	0.8
Rainy	Yes	0.9	0.1

Learning in Bayesian Networks

Bayesian networks can be constructed using two methods:

1. Structure Learning:

Identifies the dependencies (edges) between variables.

Techniques include:

- **Score-based methods**: Use scoring functions like Bayesian Information Criterion (BIC).
- **Constraint-based methods**: Use statistical tests to identify dependencies.

2. Parameter Learning:

Once the structure is fixed, parameters (CPTs) are estimated.

Methods:

Maximum Likelihood Estimation (MLE): Estimates parameters based on observed data.

Bayesian Estimation: Updates beliefs about parameters using prior distributions.

Inference in Bayesian Networks

Inference refers to querying the network to compute probabilities, either for prediction or diagnosis. Common types of inference:

Marginal Inference:

Computing the probability of a single variable irrespective of others.

Example: P(A).

Conditional Inference:

Computing the probability of one variable given evidence about others.

Example: P(A|B=b).

Most Probable Explanation (MPE):

Identifies the most likely state of all variables given evidence.

Maximum A Posteriori (MAP):Identifies the most likely value of a subset of variables given evidence.

Example:
Given evidence W=Rainy, A=Yes, find P(T=Yes).
Use **Bayes' Rule** and CPTs:

$$P(T = Yes|W = Rainy, A = Yes) = \frac{P(W)P(A)P(T|W, A)}{P(W, A)}$$

Types of Inference:
Marginal: P(T)
Conditional: P(T|W)
MPE: Most likely explanation for evidence.

Applications of Bayesian Networks
Medical Diagnosis: Predict diseases based on symptoms.
Fraud Detection: Detect anomalies in financial transactions.
Natural Language Processing: Word-sense disambiguation.
Decision Support Systems: Aid in complex decision-making under uncertainty.

Advantages
Handles uncertainty effectively.
Combines prior knowledge with observed data.
Efficient for probabilistic reasoning and inference.

Disadvantages
Computationally expensive for large networks.
Requires expertise for structure and parameter learning.

Q2) Explain naïve bays algorithm? Write any four applications of naïve Bayes classifier.
The Naïve Bayes algorithm is a probabilistic classification algorithm based on Bayes' Theorem. It assumes that features are independent of each other, which is often not true in real-world scenarios but simplifies computations. This assumption of feature independence is the "naïve" aspect of the algorithm.
Bayes' Theorem
The formula for Bayes' Theorem is:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:
P(C|X): Posterior probability of class C given predictor X.
P(X|C): Likelihood of predictor X given class C.
P(C): Prior probability of class C.
P(X): Prior probability of predictor X.
Working of Naïve Bayes
1. Training Phase:
The algorithm calculates the prior probabilities (P(C) for each class.
It calculates the likelihood (P(X|C) for each feature X given the class C.
2. Prediction Phase:
For a new data point, it computes the posterior probability for each class using Bayes' Theorem.
The class with the highest posterior probability is selected as the predicted class.
Example
Classify whether an email is "Spam" or "Not Spam" based on features like "Contains the word free," "Contains the word lottery," and "Contains the word urgent."

Training Data:

Email ID	Free	Lottery	Urgent	Class
1	Yes	Yes	No	Spam
2	No	No	Yes	Not Spam
3	Yes	No	Yes	Spam
4	No	No	No	Not Spam

Steps:

1. Compute prior probabilities:
   - $P(Spam) = 2/4 = 0.5$
   - $P(NotSpam) = 2/4 = 0.5$
2. Compute likelihoods:
   - $P(Free = Yes|Spam) = 1/2 = 0.5$
   - $P(Urgent = Yes|Spam) = 1/2 = 0.5$, etc.
3. Classify a new email with $Free = Yes, Lottery = No, Urgent = Yes$:
   - Compute $P(Spam|X)$ and $P(NotSpam|X)$, choose the higher value.

Applications of Naïve Bayes

1. Email Spam Filtering: Classify emails as spam or not spam based on words in the email.
2. Sentiment Analysis: Analyze the sentiment (positive or negative) of a product review or tweet.
3. Medical Diagnosis: Predict diseases based on symptoms (e.g., likelihood of flu given fever and cough).
4. Text Classification: Categorize documents into topics (e.g., sports, politics, technology).
5. Fraud Detection: Detect fraudulent transactions based on patterns and behavior.

Advantages of Naïve Bayes Classifier:

Simple and easy to implement.
Fast training and prediction.
Works well with high-dimensional data.
Performs well with small datasets.
Provides probabilistic predictions.
Handles both continuous and categorical data.

Disadvantages of Naïve Bayes Classifier:

Assumes feature independence, which may not always hold.
Performs poorly with highly correlated features.
Struggles with complex tasks and insufficient data.
Sensitive to imbalanced data.
May produce negative probabilities with incorrect distributions.

Q3) Explain ID-3 decision tree algorithm in detail with example.

What is a Decision Tree?

A decision tree is a supervised machine learning algorithm used for both classification and regression tasks. It models decisions and their possible consequences, including chance event outcomes, resource costs, and utility. A decision tree is a tree-like structure where:

- Each internal node represents a decision based on the value of an attribute.
- Each branch represents the outcome of the decision (such as "Yes" or "No").
- Each leaf node represents a class label or a numerical value (in the case of regression).

Types of Decision Trees:

Classification Trees: Used for classification tasks (e.g., predicting categories like "spam" or "not spam").
Regression Trees: Used for regression tasks (e.g., predicting a continuous value like house price).

ID3 (Iterative Dichotomiser 3) is a popular decision tree algorithm used for classification tasks. It works by recursively partitioning a dataset based on the attribute that provides the most information gain.

Steps involved in ID3:

1. **Create a root node:** This node represents the entire dataset.
2. **Iterative Partitioning:**

Information Gain Calculation: For each attribute, calculate the information gain, which measures how much information the attribute provides about the target class.

Attribute Selection: Select the attribute with the highest information gain.

Tree Branching: Split the node into child nodes based on the selected attribute's values.

3. **Repeat the Process:**

For each child node, repeat steps 2 and 3 until:

All instances in the node belong to the same class (leaf node).

There are no more attributes to split on.

Key Points:

Information Gain: This metric is used to determine the best attribute for splitting the data. It measures the reduction in uncertainty about the target class after splitting the dataset based on a particular attribute.

Stopping Criteria: The algorithm stops when all nodes are pure (contain only instances of the same class) or when there are no more attributes to split on.

Overfitting: ID3 can be prone to overfitting, especially when the tree becomes too complex. Techniques like pruning can be used to mitigate this issue.

Advantages:

Simple to understand and implement.

Handles both numerical and categorical data.

Can create easily interpretable decision trees.

Disadvantages:

Prone to overfitting.

Can be sensitive to noise in the data.

Assumes that attributes are independent, which may not always be the case.

In summary, the ID3 algorithm is a fundamental decision tree algorithm that has been widely used in various classification tasks. It provides a simple yet effective way to build decision trees that can be used for making predictions and understanding the relationships between features and the target variable.

Example:

Classify whether a person will play outside based on Weather conditions.

Weather	Humidity	Play Outside
Sunny	High	No
Sunny	Normal	Yes
Overcast	High	Yes
Rain	Normal	Yes
Rain	High	No

Step 1: Calculate Entropy of the Target (Play Outside)

We have:

- 3 Yes and 2 No.

$$E(S) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$$

Step 2: Calculate Information Gain for Each Attribute

For "Weather":

1. **Sunny**: $2\,\text{Samples} \rightarrow 1\,\text{Yes}, 1\,\text{No}$

$$E(Sunny) = -\left(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}\right) = 1.0$$

2. **Overcast**: $1\,\text{Sample} \rightarrow 1\,\text{Yes}$

$$E(Overcast) = 0 \quad \text{(Pure Node)}$$

3. **Rain**: $2\,\text{Samples} \rightarrow 1\,\text{Yes}, 1\,\text{No}$

$$E(Rain) = -\left(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}\right) = 1.0$$

Weighted Average Entropy for Weather:

$$E(Weather) = \frac{2}{5}(1.0) + \frac{1}{5}(0) + \frac{2}{5}(1.0) = 0.8$$

Information Gain:

$$\text{Gain(Weather)} = 0.971 - 0.8 = 0.171$$

For "Humidity":

1. **High**: $3\,\text{Samples} \rightarrow 1\,\text{Yes}, 2\,\text{No}$

$$E(High) = -\left(\frac{1}{3}\log_2\frac{1}{3} + \frac{2}{3}\log_2\frac{2}{3}\right) = 0.918$$

2. **Normal**: $2\,\text{Samples} \rightarrow 2\,\text{Yes}$

$$E(Normal) = 0 \quad \text{(Pure Node)}$$

Weighted Average Entropy for Humidity:

$$E(Humidity) = \frac{3}{5}(0.918) + \frac{2}{5}(0) = 0.550$$

Information Gain:

$$\text{Gain(Humidity)} = 0.971 - 0.550 = 0.421$$

Step 3: Choose the Best Attribute

Humidity has the highest Information Gain (0.421).

Split the dataset on "Humidity."

Step 4: Recursive Splitting

After Splitting on "Humidity":

1. High: Further split using Weather.
2. Normal: All are "Yes" → Stop.

Splitting "High" using Weather:

Sunny: 1 Sample→ No → Stop.

Overcast: 1 Sample→ Yes → Stop.

Rain: 1 Sample→ No → Stop.

Final Decision Tree

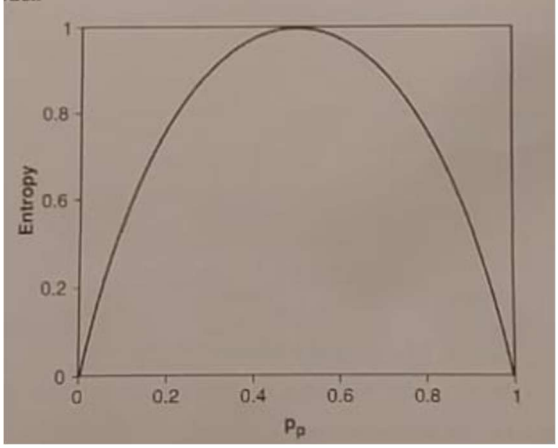
```yaml
                Humidity
            /           \
        High            Normal
    /    |    \            \
Sunny Overcast Rain        Yes
  No     Yes     No
```

Q4) Explain the following measures of impurity with example i) Information Gain ii) Gini Index iii) Entropy

Entropy

In a decision tree context, Entropy measures randomness or impurity of an attribute (or datapoint) in a decision tree. Entropy generally lies between 0 and 1. A lower value of entropy is desired. A lower value of entropy signifies a homogeneous dataset with less randomness (hence better predictions). A high entropy indicates high disorder. Entropy could also be more than 1. It then signifies that the dataset is very random and is not good for creating a prediction or classification model.

As you see in the entropy graph:



A probability of 0.5 makes entropy equal to 1 which means equally divided samples (not good for predicting)
A probability of 1 makes entropy 0 which means prediction with 100% accuracy
The goal is to have low entropy to ensure that you can predict the event either happening or not happening

Entropy is calculated using the following formula: $\text{Entropy } H_x = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i)$

Where x is the random variable with n outcomes. The log in the formula is taken as log to the base 2 ($\log_2$). p(x) is the probability or distribution of the input variable.

Example:

Consider the following dataset:

Weather	Play Outside
Sunny	No
Sunny	Yes
Overcast	Yes
Rain	Yes
Rain	No

Using the same dataset:

3 Yes, 2 No:

$$E(S) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$$

Entropy is directly used in **Information Gain** as shown above.

Information Gain:

Information gain is a measure of purity produced by an attribute.
It is a representation of how well the given attribute separates out the given dataset based on the desired target classification.
Information gain is high if the chosen attribute produces near pure subset of datapoints (subset containing majority of one class of datapoints) after split.
Information gain is low if the chosen attribute does not produce near pure subset of datapoints (subset containing majority of one class of datapoints) after split.
To calculate how impure a split subset is, you calculate its entropy.
Information gain is calculated as: IG = Entropy (before split) - Entropy (after split for all subsets)

At each step in constructing the decision tree, you choose to split the data on the attribute with the highest value in information gain as this leads to the purest subsets. Information gain compares the degree of purity of the parent node before a split with the degree of purity of the child node after a split. At each split, an attribute with the greatest information gain is considered the most informative attribute.
Example:

Step 1: Entropy of Target (Play Outside):

- 3 Yes, 2 No:

$$E(S) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$$

Step 2: Split by Weather:

1. Sunny: $2 \rightarrow 1\,\text{Yes}, 1\,\text{No}$

$$E(Sunny) = -\left(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}\right) = 1.0$$

2. Overcast: $1 \rightarrow 1\,\text{Yes}$

$$E(Overcast) = 0$$

3. Rain: $2 \rightarrow 1\,\text{Yes}, 1\,\text{No}$

$$E(Rain) = 1.0$$

Weighted Entropy of Weather:

$$E(Weather) = \frac{2}{5}(1.0) + \frac{1}{5}(0) + \frac{2}{5}(1.0) = 0.8$$

Information Gain:

$$\text{IG} = 0.971 - 0.8 = 0.171$$

Gini Index:

Gini index is another purity measurement method used for splitting the datapoints in a decision tree. It only works with categorical variables (Successes or Failures).

Gini index is calculated by squaring the probabilities of "successes" and "failures" and finding the datapoints in the subsets after the split.

It measures the size of datapoints in the parent node before the split and the homogeneity of the sub-nodes.

Gini for sub - nodes $=(p_i)^2$

Gini Index $= \Sigma\, w_i\, (p_i)^2$

Where p_i is the probability of the datapoints in a set and w_i is the weight of the sub-node after the split. k is the number of sub-nodes formed.

There is also a related term called Gini impurity which is given as:

Gini Impurity = 1 - Gini Index

Higher the value of the Gini index, higher is the impurity.

Example:

1. Before Splitting (Parent Node):

$3\,\text{Yes}, 2\,\text{No}$:

$$Gini(S) = 1 - \left(\left(\frac{3}{5}\right)^2 + \left(\frac{2}{5}\right)^2\right) = 1 - (0.36 + 0.16) = 0.48$$

2. After Splitting on Weather:

- Sunny ($2 \rightarrow 1\,\text{Yes}, 1\,\text{No}$):

$$Gini(Sunny) = 1 - \left(\left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2\right) = 0.5$$

- Overcast ($1\,\text{Yes}$):

$$Gini(Overcast) = 0$$

- Rain ($2 \rightarrow 1\,\text{Yes}, 1\,\text{No}$):

$$Gini(Rain) = 0.5$$

Weighted Gini Index for Weather:

$$Gini(Weather) = \frac{2}{5}(0.5) + \frac{1}{5}(0) + \frac{2}{5}(0.5) = 0.4$$

Gini Gain:

$$Gini\ \text{Gain} = 0.48 - 0.4 = 0.08$$

Q5)